Practical exercise 1: Algorithm A^*

1. Problem statement

The objective of this exercise, to be developed along two sessions, is to understand and implement the algorithm A^* for searching in directed graphs. The algorithm should be implemented in the most general possible way, but it will be tested by applying it to a game called Takuzu or Binairo. This game can be described as follows:

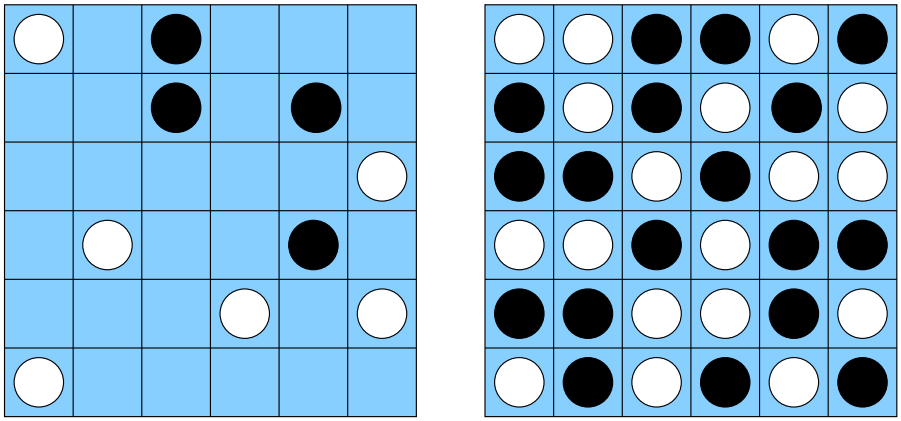
Let there be a square board of $n \times n$ squares where n is an even number. Each square of the board may contain a white piece, a black piece, or may be empty. The game starts with a given initial configuration with some pieces already placed that comply with the rules that we will detail below, and the player (there is only one player) must place pieces, of any color and in any square of his/her choice, so that **at all times** the following rule is fulfilled:

> 1) There cannot be more than two consecutive squares, neither in a row nor in a column, that have pieces of the same color.

and the final configuration must also comply with these rules:

> 2) No square can be left empty.
> 3) In each row and in each column there must be the same number of black and white squares (obviously, $\frac{n}{2}$).
> 4) No row or column can be repeated (i.e.: two different rows or columns cannot have the same pieces in the same order).

An example of a Takuzu from 6×6 in its initial and final states would be:



Note that the player cannot place a piece of color 'c', say, anywhere he wants, i.e. not all moves are legal: some would violate rule 1, by placing three consecutive pieces of color 'c' in a row or column. Others would violate rule 3, if there are already $\frac{n}{2}$ pieces of color 'c' in that row/column; and if a single piece is missing to fill a row or column, completing it so that the number of pieces is as requested could break rule 4.

Also, note that placing a piece in a legal position **forces** other pieces to be placed: if a piece of color 'c' has another of the same color next to it, any subsequent empty square will have to be filled with a piece of the opposite color. Similarly, if there are already $\frac{n}{2}$ pieces of color 'c' in a row/column, all empty squares in that row/column must be filled with the opposite color. We will call these **compulsory moves** and the program will perform them automatically.

In our formalism each state of the board will be a **node** of the state graph. The neighbors of that node will be all the states generated by placing a piece in any of the **possible legal** positions **and then, executing all the compulsory moves**. Thus, the neighbor state to a given one in which there was p pieces placed will not necessarily have $p + 1$ pieces, but probably some or many more.

The program proceeds by generating all the neighbor nodes of the current one (in the sense of the previous paragraph), evaluating each one with the functions f and g whose meaning was given in class for the algorithm A^*, and applying that algorithm to find the target node, which is the one in which the complete board is full. Your task is to program the algorithm A^* and test it with various heuristic functions $h(n)$. h should always give a lower value to those states of the board that you consider to be better (more desirable).

2. Implementation

2.1. Data format

Some Takuzus are provided for testing, which are given as text files, of progressively larger sizes. Those named `tnne` are of size nn and easy ('easy'); those named `tnnh` are more difficult ('hard').

To starting the game the internal program will read these files and transform them into its internal representation (an object of the class `Board`, see appendix). From there, you have to solve the problem using A^*, which you will program as a function. Your A^* will generate the solution and display it on the screen along with the number of iterations used to reach it.

3. Delivery

The submission will be delivered in a compressed file containing:

- The code SUFFICIENTLY COMMENTED. Also include the Makefile to compile it, which is already provided.
- A README text file with instructions for compiling (generally, just running make), some comments comparing the results with the different heuristics you have programmed, and any additional comments you want to make.

WARNING: Practicals that do not even compile will not be marked (and will therefore be invalid). If they compile (in the sense of generating an executable) but the executable does nothing that makes sense, the practice will probably get a fail.

4. Appendix: C++ API

In the file `takuzu.h` are defined:

- The data type defined by enumeration `Token={e,b,w}` to indicate the possible content of a box (empty, black, white).
- The data type `Position` which is a structure with two fields of type `unsigned` called `row` and `col`, to represent the coordinates of a square, which are counted from 0.
- The data type `Move` which is a structure with two fields, one of type `Position` called `sq` and another of type `Token` called `value`, to represent the movement consisting of placing a `value` colored token in the `sq` square.
- The `Board` class to represent a board state (a node) and in it two data structures:
  `typedef list<Board> lb`
  to represent a list of boards, that is, nodes.
  `typedef list<pair<Move,Board> > lnei`
  to represent a list of nodes neighboring another, where the first element of the pair represents the movement that will take us to a certain state, and the second the state we reach by actually performing that movement **and its subsequent compulsory moves**.

- The `Scores` class, to store the values of the `f` and `g` functions of any board.

The public constructors and functions (the only ones you can use) of the `Board` class are:

`Board(unsigned size)` Constructor that creates an empty board with `size` squares on each side. `size` must be an even number.

`Board(string file_name)` Constructor that creates a board from the text file `file_name`, in the format of the examples provided.

`Board(string file_name,ostream &outst)` As above, but after loading the file it displays on the screen (if `cout` is passed) or writes to an open text file (if the file descriptor is passed) the initial state, and the state after making the compulsory moves.

`unsigned GetSize()` Function that returns the number of squares on each side of the board.

`Token Content(Position p)` Function that returns the contents of the square at position p.

`void Show(ostream &outst)` Procedure that displays on the screen (passing `cout`) or writes to a text file (passing the descriptor of the open file) the board in its current state.

`void GetNeighbours(lnei &listnei)` Procedure that fills the variable `listnei` passed by reference with a list of resulting Move/Board pairs for all legal moves that can be made from the current state. As stated, the board in the list results from having executed not only the move, but all the moves required by it.

`bool Member(lb &listboards)` Function that returns `true` or `false` depending on whether the current board is or is not in the list of boards that is passed as an argument.

On the other hand, the constructors and public functions (the only ones you can use) of the `Scores` class are:

`void SetfValue(Board b,float v)` Procedure that stores in the table of the `f` function the given value for the board passed to it.

`float GetfValue(Board b)` Function that returns the currently stored value of `f` for the board passed to it, or $+\infty$ (in practice, the constant `FLT_MAX`) if no value has yet been set for that board.

`void SetgValue(Board b,float v)` Procedure that stores in the function table `g` the given value for the board passed to it.

`float GetgValue(Board b)` Function that returns the currently stored value of `g` for the board passed to it passes, or $+\infty$ (in practice, the constant `FLT_MAX`) if no value has been set for that board yet.

A typical test program would be:

```
./takuzu_test t06h
```

which would be called from the command line passing as an argument the input file containing the proposed takuzu.

You must implement the function `bool AStar(...)` (think about what parameters should be passed to it) which will return `true` or `false` depending on whether there is a solution or not. You must also implement any auxiliary functions you consider necessary All this can simply go in the main file, before `main`.

And finally, the heuristics must be implemented in the file `heuristic.cpp`, whose prototype must in any case be the one indicated in `heuristic.h`, that is:

```
float h(Board current_board,Move m,Board next_board)
```

Within the function, it is not obligatory to use its three parameters; it depends on the heuristic you have thought of. To write several heuristics and test them successively, you can write different files `heuristic.cpp`, or put them all in the same, using conditional directives of the form

```
#ifdef name_HEURISTIC
(write the h..)
#endif
```

and using the name you gave to the heuristic as the value of the variable `CHOICE` that appears at the beginning of the `Makefile`.